المدرسة الوطنية للعلوم التطبيقية - بني ملال

ⵜⴰⵙⴷⴰⵡⵉⵜ ⵏ ⵜⵎⴰⵙⵙⴰⵏⵉⵏ ⵜⵉⵏⵎⵍⴰⵍⵉⵏ - ⴱⵏⵉ ⵎⵍⴰⵍ

Ecole Nationale des Sciences Appliquées - Béni Mellal

COMPTE RENDU DU TP-4

1ÈRE ANNÉE – FILIÈRE INTELLIGENCE ARTIFICIELLE ET CYBERSÉCURITÉ

MODULE : ARCHITECTURE LOGICIELLE

Réalisé par :

ASIM ZINEB

Encadré par :

Pr. BE ELBAGHAZAOUI

Année unniversitaire:2025_2026

Objectifs

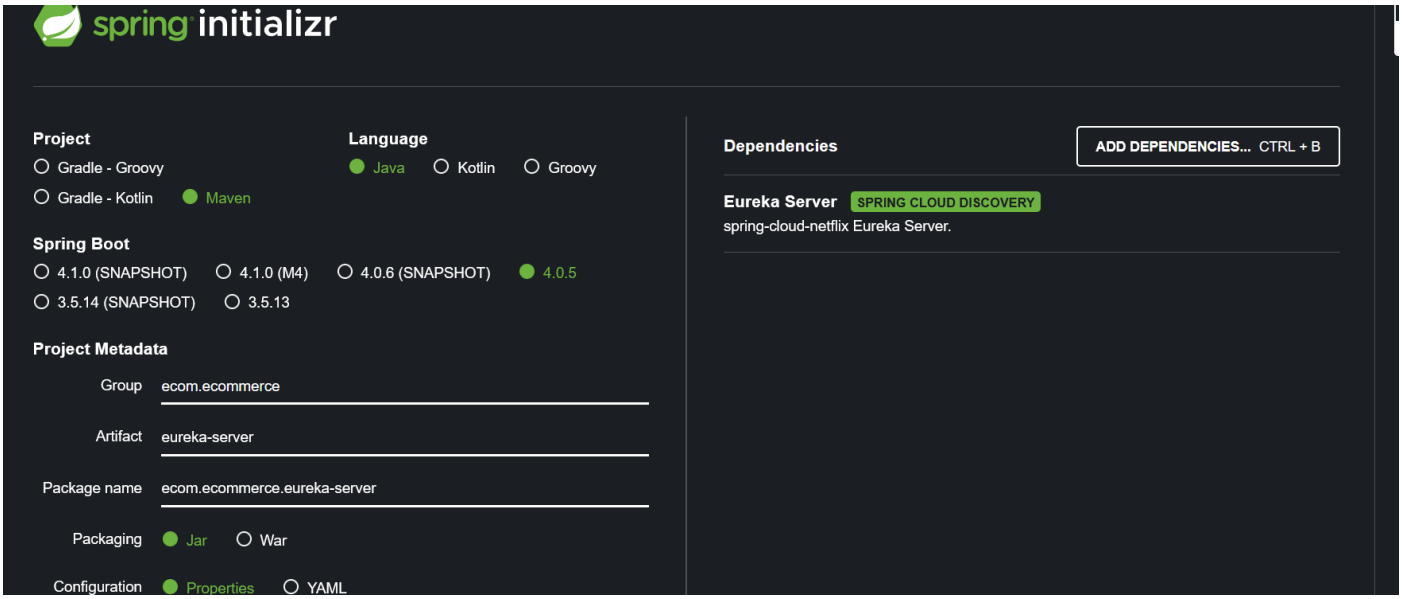
Ce TP vise à mettre en place une architecture orientée services (SOA) avec Spring Boot en créant des microservices indépendants qui communiquent entre eux via REST. Il s'agit de comprendre le rôle d'un service de découverte Eureka, d'implémenter la communication inter-services avec OpenFeign, et de tester le tout avec Postman.

Outils utilisés

- Java JDK
- Maven IntelliJ IDE
- Postman
- Eureka Server

Etape 1 :

- **Création du projet**



- **Configuration**

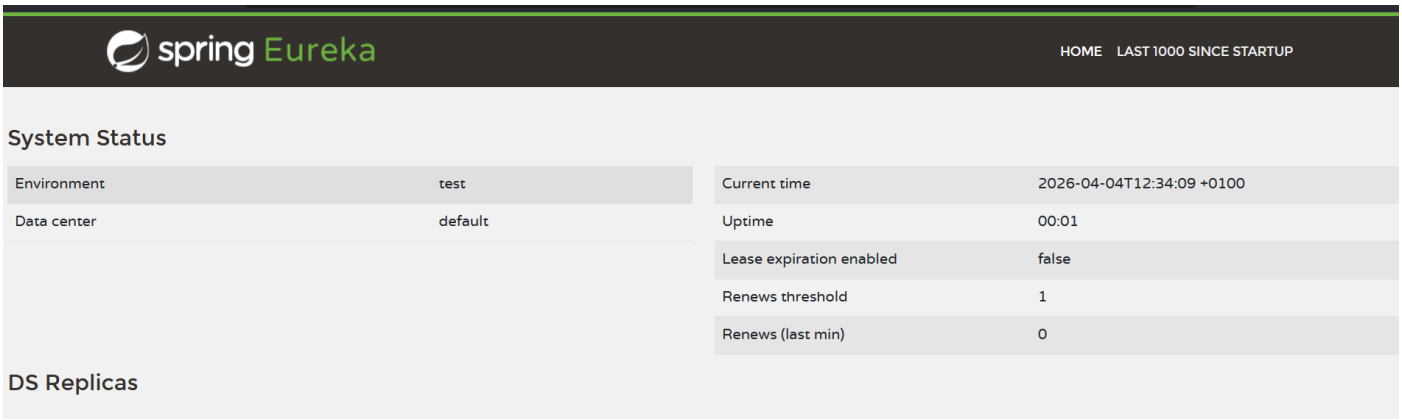
```
application.properties   EurekaServerApplication.java
1  spring.application.name=eureka-server
2  server.port=8761
3  eureka.client.register-with-eureka=false
4  eureka.client.fetch-registry=false
5  eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

- **Classe principale**

```
application.properties   EurekaServerApplication.java
1  package com.ecommerce.eureka_server;
2  import org.springframework.boot.SpringApplication;
3  import org.springframework.boot.autoconfigure.SpringBootApplication;
4  import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;
5  @SpringBootApplication
6  @EnableEurekaServer
7  public class EurekaServerApplication {
8      public static void main(String[] args) { SpringApplication.run(EurekaServerApplication.class, args); }
11 }
```

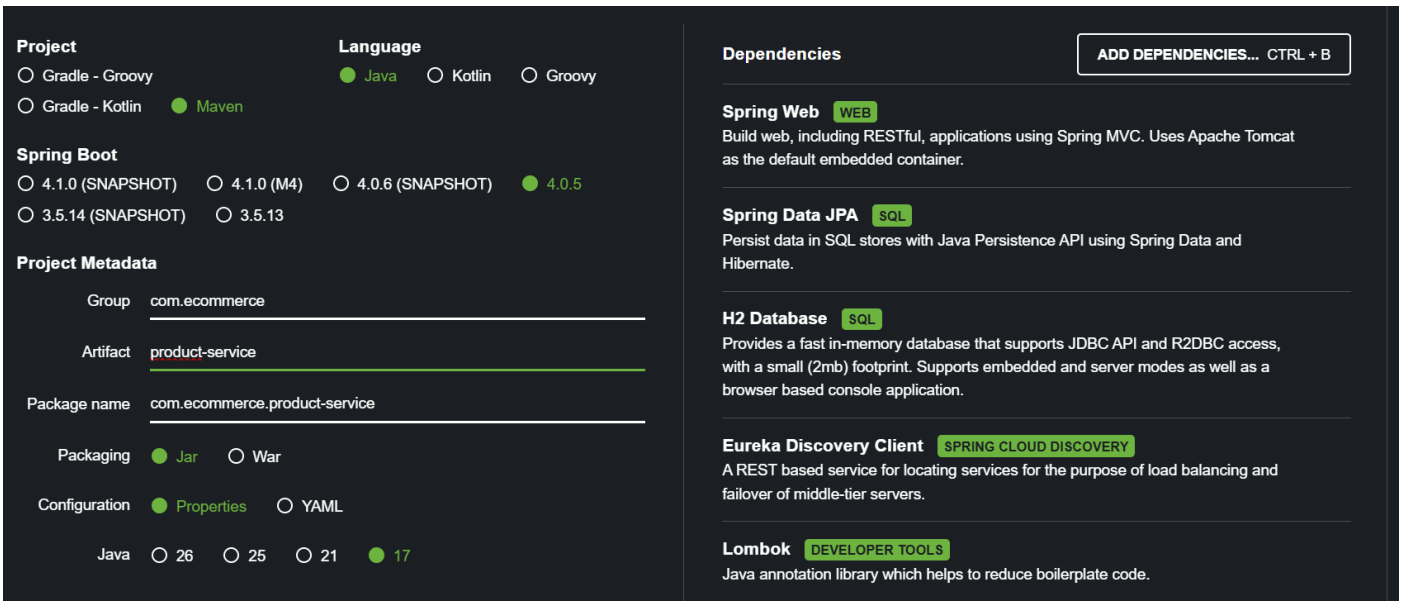
- **Démarrage**

Le serveur a été lancé avec la commande **mvn spring-boot:run** et est accessible sur **http://localhost:8761**. Le tableau de bord Eureka confirme que le serveur est opérationnel. Aucun service n'est encore enregistré à ce stade.



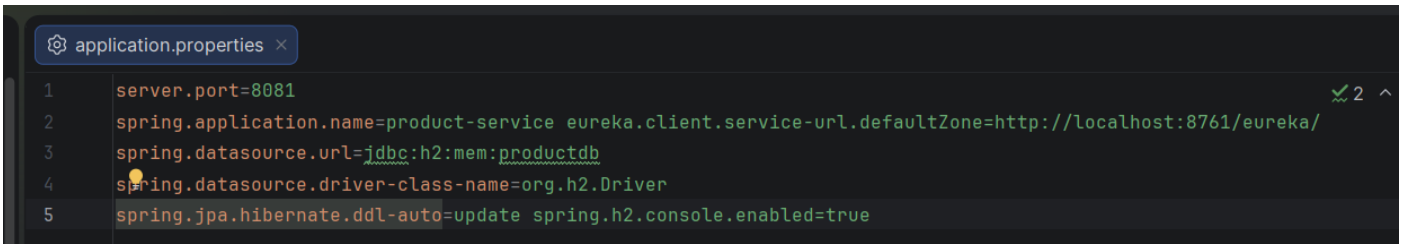
Etape 2 : Product Service

- **Création du projet**



- **Configuration**

Fichier application.properties :



```
application.properties
server.port=8081
spring.application.name=product-service eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
spring.datasource.url=jdbc:h2:mem:productdb
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=update spring.h2.console.enabled=true
```

- **Modèle de données**

La classe Product.java

```java
package com.ecommerce.productservice.model;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String description;
    private Double price;
    private Integer stock;
}
```

- **Repository**

ProductRepository.java

```java
application.properties   Product.java   ProductRepository.java ×   ProductService.java   pom.xml (product-servic
1  package com.ecommerce.product_service.repository;
2  import com.ecommerce.product_service.model.Product;
3  import org.springframework.data.jpa.repository.JpaRepository;
4  import org.springframework.stereotype.Repository;
5  @Repository  2 usages
6  public interface ProductRepository extends JpaRepository<Product, Long> { }
```

- **Service**

ProductService.java

```java
ProductService.java ×   application.properties   Product.java   ProductRepository.java   ProductServiceApplication.jav
1  package com.ecommerce.productservice.service;                                                          4
2  import com.ecommerce.productservice.model.Product;
3  import com.ecommerce.productservice.repository.ProductRepository;
4  import lombok.RequiredArgsConstructor;
5  import org.springframework.stereotype.Service;
6  import java.util.List; import java.util.Optional;
7  @Service
8  @RequiredArgsConstructor
9  public class ProductService {
10     private final ProductRepository productRepository;
11     public List<Product> getAllProducts() {         return productRepository.findAll();     }  no usages
12     public Optional<Product> getProductById(Long id) {         return productRepository.findById(id);     }  no usage
13     public Product createProduct(Product product) {         return productRepository.save(product);     }  no usages
14     public void deleteProduct(Long id) {         productRepository.deleteById(id);     }  no usages
15 }
```

- **Contrôleur REST**

ProductController.java

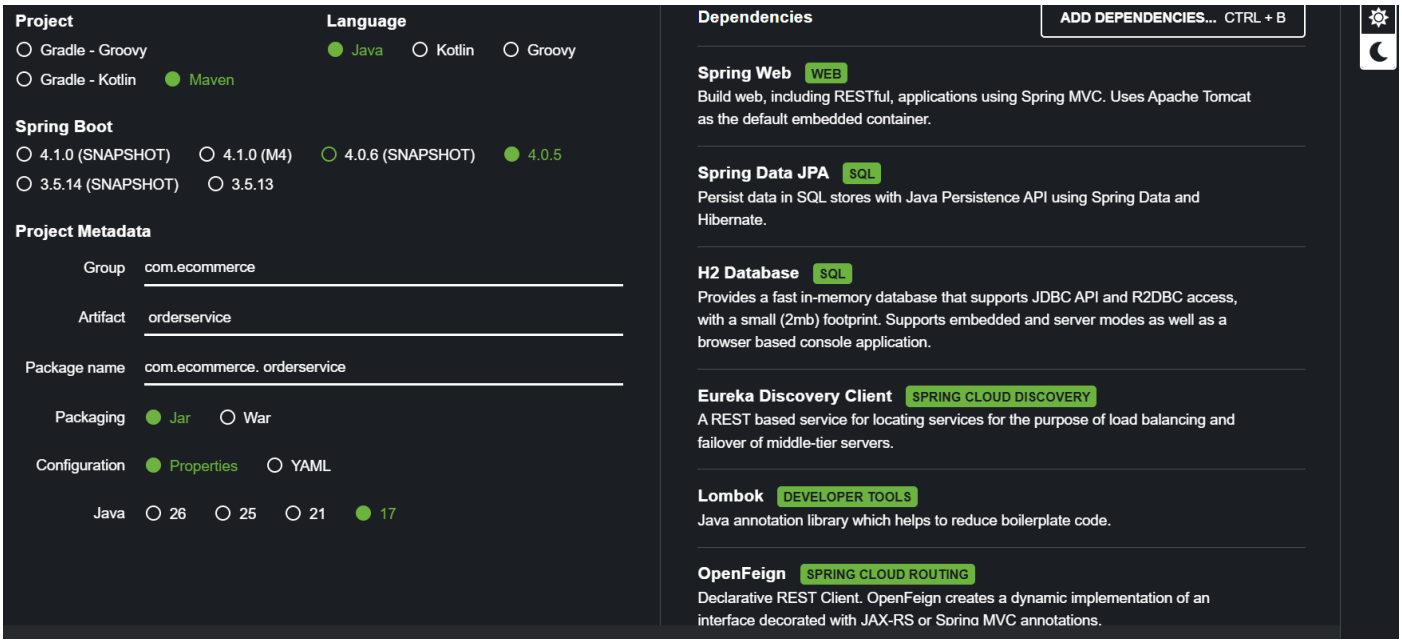
```java
package com.ecommerce.productservice.controller;
import com.ecommerce.productservice.model.Product;
import com.ecommerce.productservice.service.ProductService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;
@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductController {
    private final ProductService productService;
    @GetMapping public List < Product > getAllProducts() {
        return productService.getAllProducts();
    }
    @GetMapping("/{id}") public ResponseEntity < Product > getProductById(@PathVariable Long id) {
        return productService.getProductById(id).map(ResponseEntity::ok).orElse(ResponseEntity.notFound().build());
    }
    @PostMapping public Product createProduct(@RequestBody Product product) {
        return productService.createProduct(product);
    }
    @DeleteMapping("/{id}") public ResponseEntity < Void > deleteProduct(@PathVariable Long id) {
        productService.deleteProduct(id);
        return ResponseEntity.noContent().build();
    }
}
```

- **Activation Eureka Client**

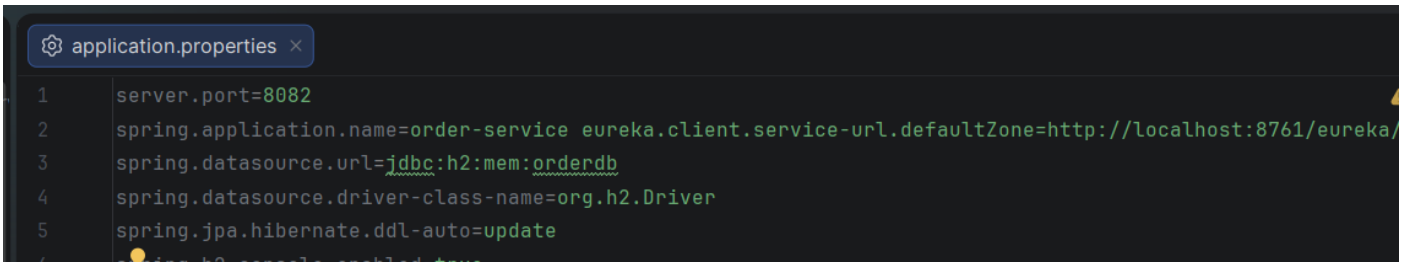
```java
package com.ecommerce.productservice;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.discovery.EnableDiscoveryClient;
@SpringBootApplication
@EnableDiscoveryClient
public class ProductServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(ProductServiceApplication.class, args);
    }
}
```

Etape 3 : service Order

- **Création du projet**



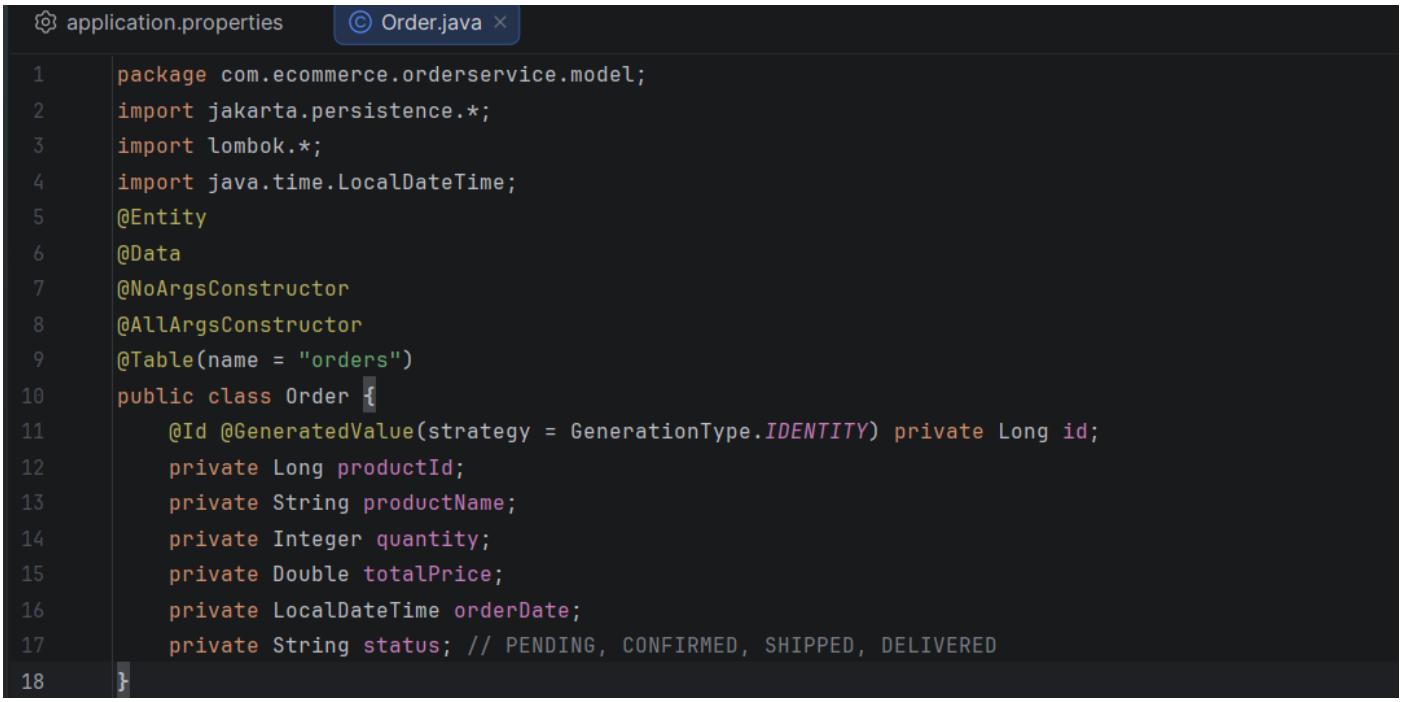
- **Configuration**



```
application.properties
server.port=8082
spring.application.name=order-service eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
spring.datasource.url=jdbc:h2:mem:orderdb
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=update
```

- **Modèle de données**

Fichier Order.java



```
application.properties   Order.java
package com.ecommerce.orderservice.model;
import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDateTime;
@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
@Table(name = "orders")
public class Order {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
    private Long productId;
    private String productName;
    private Integer quantity;
    private Double totalPrice;
    private LocalDateTime orderDate;
    private String status; // PENDING, CONFIRMED, SHIPPED, DELIVERED
}
```

- **DTO pour Product**

ProductDTO.java

```java
package com.ecommerce.orderservice.dto;
import lombok.*;
@Data  2 usages
@NoArgsConstructor
@AllArgsConstructor
public class ProductDTO {
    private Long id;
    private String name;
    private String description;
    private Double price;
    private Integer stock;
}
```

- **Client Feign pour Product Service**

ProductClient.java pour communiquer avec Product Service

application.properties | Order.java | ProductClient.java | OrderRepository.java | OrderService.java | OrderC

```java
package com.ecommerce.orderservice.client;
import com.ecommerce.orderservice.dto.ProductDTO;
import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
@FeignClient(name = "productservice")  2 usages
public interface ProductClient {
    @GetMapping("/api/products/{id}") ProductDTO getProductById(@PathVariable Long id);  1 usage
}
```

- **Repository et Service**

OrderRepository.java

application.properties | Order.java | ProductClient.java | OrderRepository.java | ProductDTO.java

```java
package com.ecommerce.orderservice.Repository;
import com.ecommerce.orderservice.model.Order;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
@Repository  no usages
public interface OrderRepository extends JpaRepository < Order, Long > {}
```

OrderService.java

```java
package com.ecommerce.orderservice.service;
import com.ecommerce.orderservice.client.ProductClient;
import com.ecommerce.orderservice.dto.ProductDTO;
import com.ecommerce.orderservice.model.Order;
import com.ecommerce.orderservice.Repository.OrderRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import java.time.LocalDateTime;
import java.util.List;
@Service
@RequiredArgsConstructor
public class OrderService {
    private final OrderRepository orderRepository;
    private final ProductClient productClient;
    public List < Order > getAllOrders() {  no usages
        return orderRepository.findAll();
    }
    public Order createOrder(Long productId, Integer quantity) {  no usages

        ProductDTO product = productClient.getProductById(productId);
        Order order = new Order();
        order.setProductId(productId);
        order.setProductName(product.getName());
        order.setQuantity(quantity);
        order.setTotalPrice(product.getPrice() * quantity);
        order.setOrderDate(LocalDateTime.now());
        order.setStatus("PENDING");
        return orderRepository.save(order);
    }
}
```

- **Contrôleur**

OrderController.java

```java
package com.ecommerce.orderservice.controller;
import com.ecommerce.orderservice.model.Order;
import com.ecommerce.orderservice.service.OrderService;
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.*;
import java.util.List;
@RestController
@RequestMapping("/api/orders")
@RequiredArgsConstructor
public class OrderController {
    private final OrderService orderService;
    @GetMapping public List < Order > getAllOrders() {
        return orderService.getAllOrders();
    }
    @PostMapping public Order createOrder(@RequestParam Long productId, @RequestParam Integer quantity) {
        return orderService.createOrder(productId, quantity);
    }
}
```

- **Activation des clients Feign**

```java
package com.ecommerce.orderservice;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.discovery.EnableDiscoveryClient;
import org.springframework.cloud.openfeign.EnableFeignClients;
@SpringBootApplication
@EnableDiscoveryClient
@EnableFeignClients
public class OrderserviceApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrderserviceApplication.class, args);
    }
}
```

Étape 4 : Test l'application

- **Démarrage des services**

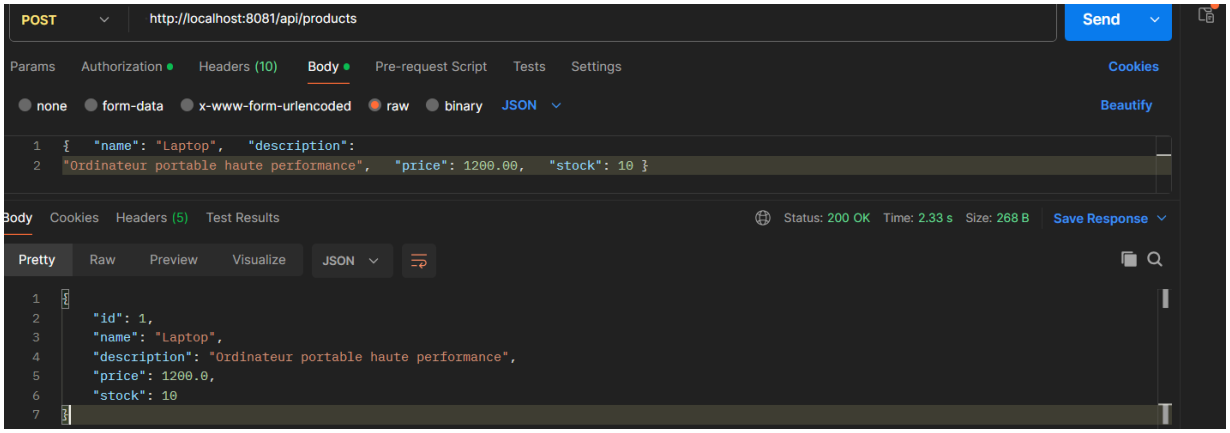
Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
ORDER-SERVICE	n/a (1)	(1)	UP (1) - localhost:order-service:8082
PRODUCTSERVICE	n/a (1)	(1)	UP (1) - localhost:productservice:8081

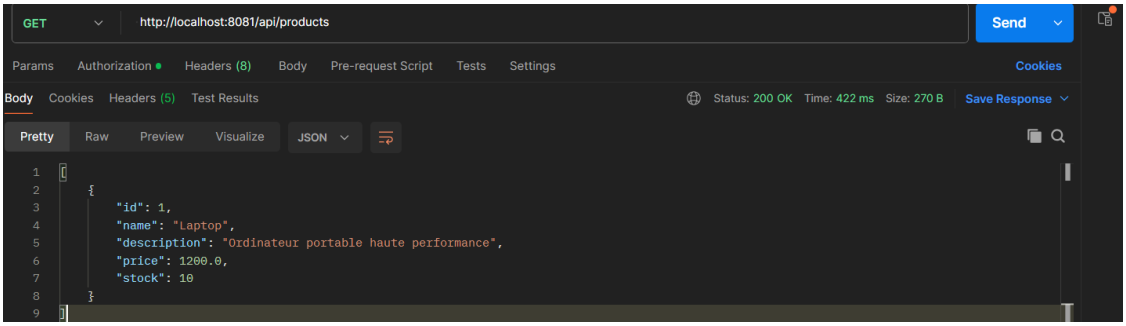
General Info

- **Tests avec Postman**

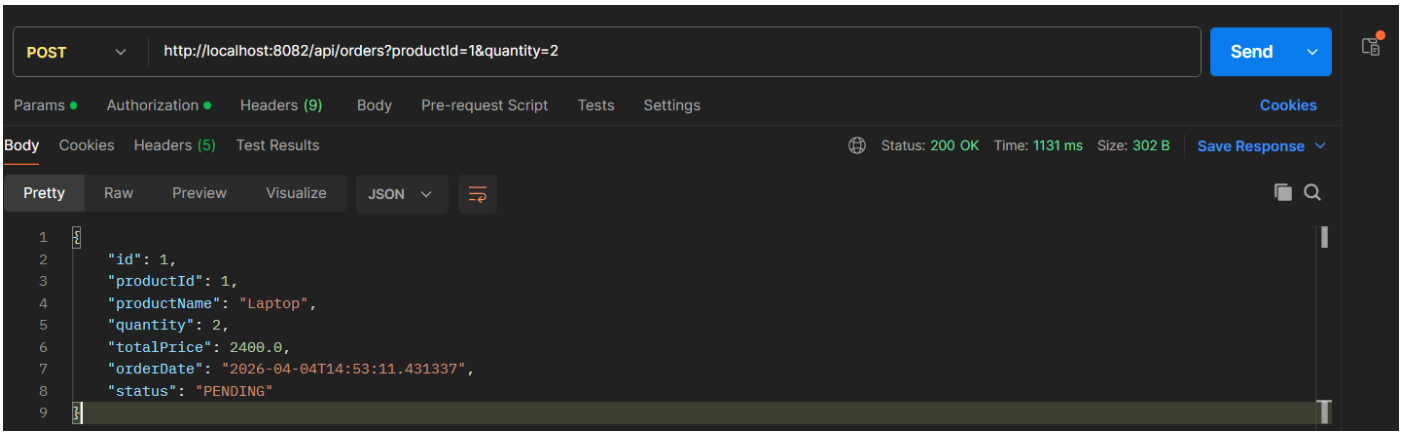
✓ **Test 1 : Créer un produit**



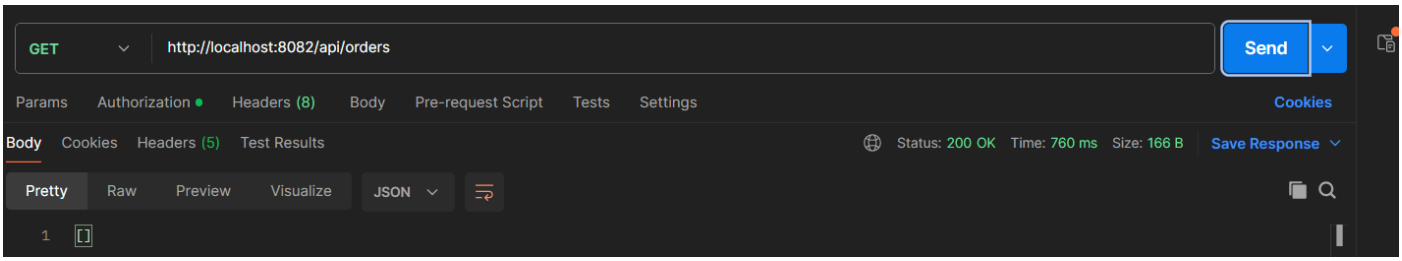
✓ **Test 2 : Lister les produits**



✓ **Test 3 : Créer une commande POST**



✓ **Test 4 : Lister les commandes GET**



Exercices pratiques

Exercice 1 : Ajout d'une fonctionnalité

On ajoute l'endpoint PUT dans ProductService, on modifie ProductController.java

```
    }
    @PutMapping("/{id}/stock")
    public ResponseEntity<Product> updateStock(@PathVariable Long id,
                                               @RequestParam Integer quantity) {
        return productService.updateStock(id, quantity)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }
}
```

On ajoute la méthode dans ProductService.java

```
public Optional<Product> updateStock(Long id, Integer quantity) {  1 usage
    return productRepository.findById(id).map( Product product -> {
        if (product.getStock() < quantity) {
            throw new RuntimeException("Stock insuffisant");
        }
        product.setStock(product.getStock() - quantity);
        return productRepository.save(product);
    });
}
}
```

On ajouter l'appel dans OrderService en ProductClient.java

```
ies    Order.java    ProductClient.java ×    OrderRepository.java    OrderService.java    OrderContr
package com.ecommerce.orderservice.client;
import com.ecommerce.orderservice.dto.ProductDTO;
import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestParam;

@FeignClient(name = "productservice")  2 usages
public interface ProductClient {
    @GetMapping("/api/products/{id}")  1 usage
    ProductDTO getProductById(@PathVariable Long id);

    @PutMapping("/api/products/{id}/stock")  1 usage
    void updateStock(@PathVariable Long id, @RequestParam Integer quantity);
}
```

On modifie OrderService.java

```
    public Order createOrder(Long productId, Integer quantity) {  1 usage
        ProductDTO product = productClient.getProductById(productId);

        if (product.getStock() < quantity) {
            throw new RuntimeException("Stock insuffisant pour ce produit");
        }
        Order order = new Order();
        order.setProductId(productId);
        order.setProductName(product.getName());
        order.setQuantity(quantity);
        order.setTotalPrice(product.getPrice() * quantity);
        order.setOrderDate(LocalDateTime.now());
        order.setStatus("PENDING");

        Order savedOrder = orderRepository.save(order);

        productClient.updateStock(productId, quantity);

        return savedOrder;
    }
}
```

Test

Stock initial

GET http://localhost:8081/api/products/1 Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

Body Cookies Headers (5) Test Results Status: 200 OK Time: 386 ms Size: 268 B Save Response

Pretty Raw Preview Visualize JSON


```
{
    "id": 1,
    "name": "Laptop",
    "description": "Ordinateur portable haute performance",
    "price": 1200.0,
    "stock": 10
}
```


Création d'une commande

POST http://localhost:8082/api/orders?productId=1&quantity=3

Params Authorization Headers (9) Body Pre-request Script Tests Settings

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON


```
{
    "id": 1,
    "productId": 1,
    "productName": "Laptop",
    "quantity": 3,
    "totalPrice": 3600.0,
    "orderDate": "2026-04-04T15:48:11.4459409",
    "status": "PENDING"
}
```


Stock après commande

GET http://localhost:8081/api/products/1

Params Authorization Headers (8) Body Pre-request Script Tests Settings

Body Cookies Headers (5) Test Results Sta

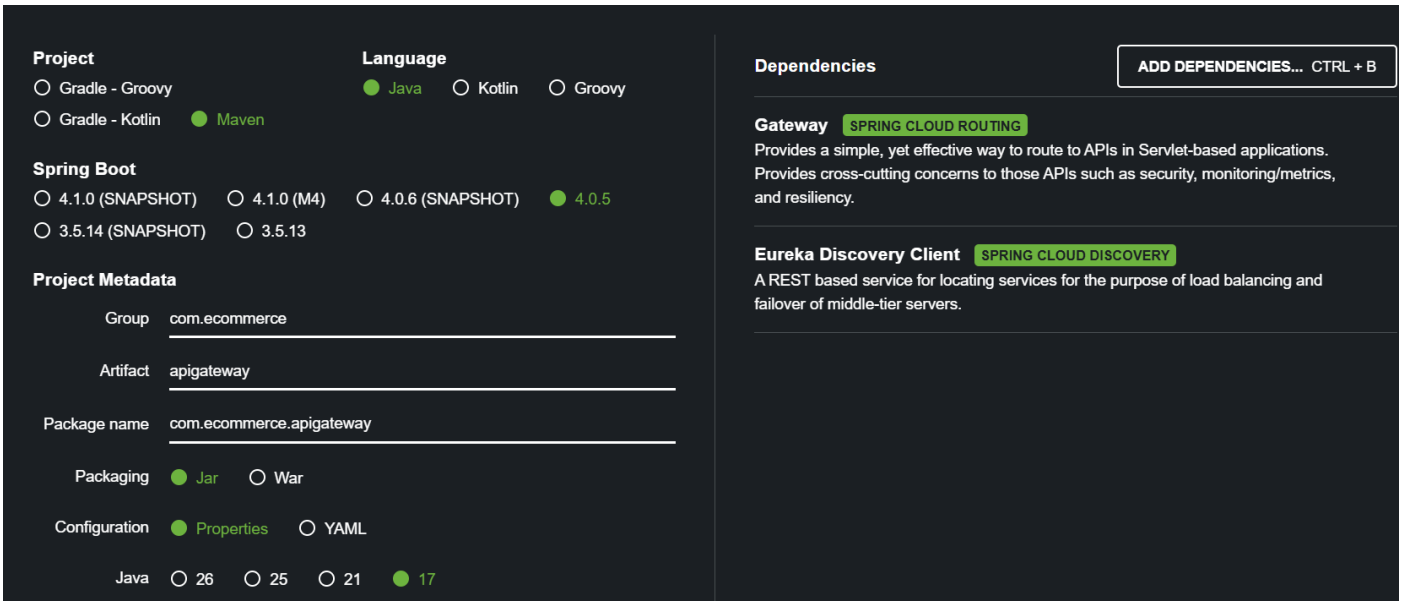
Pretty Raw Preview Visualize JSON


```
{
    "id": 1,
    "name": "Laptop",
    "description": "Ordinateur portable haute performance",
    "price": 1200.0,
    "stock": 7
}
```


Le stock est décrémenté

Exercice 2 : Création d'une API Gateway

Création du projet



Configuration

application.properties

```
spring.application.name=apigateway
server.port=8080
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
spring.cloud.gateway.routes[0].id=product-service
spring.cloud.gateway.routes[0].uri=lb://productservice
spring.cloud.gateway.routes[0].predicates[0]=Path=/api/products/**
spring.cloud.gateway.routes[1].id=order-service
spring.cloud.gateway.routes[1].uri=lb://order-service
spring.cloud.gateway.routes[1].predicates[0]=Path=/api/orders/**
```

Classe principale

```
package com.ecommerce.apigateway;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.discovery.EnableDiscoveryClient;
@SpringBootApplication
@EnableDiscoveryClient
public class ApigatewayApplication {
    public static void main(String[] args) {
        SpringApplication.run(ApigatewayApplication.class, args);
    }
}
```

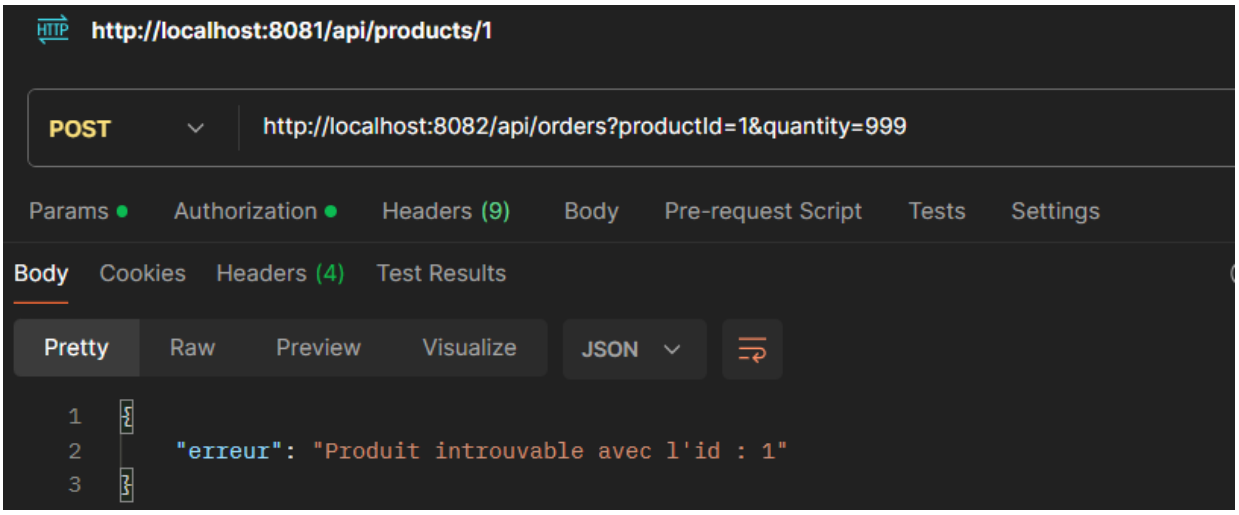
Exercice 3 : Gestion des erreurs

On modifier OrderService.java

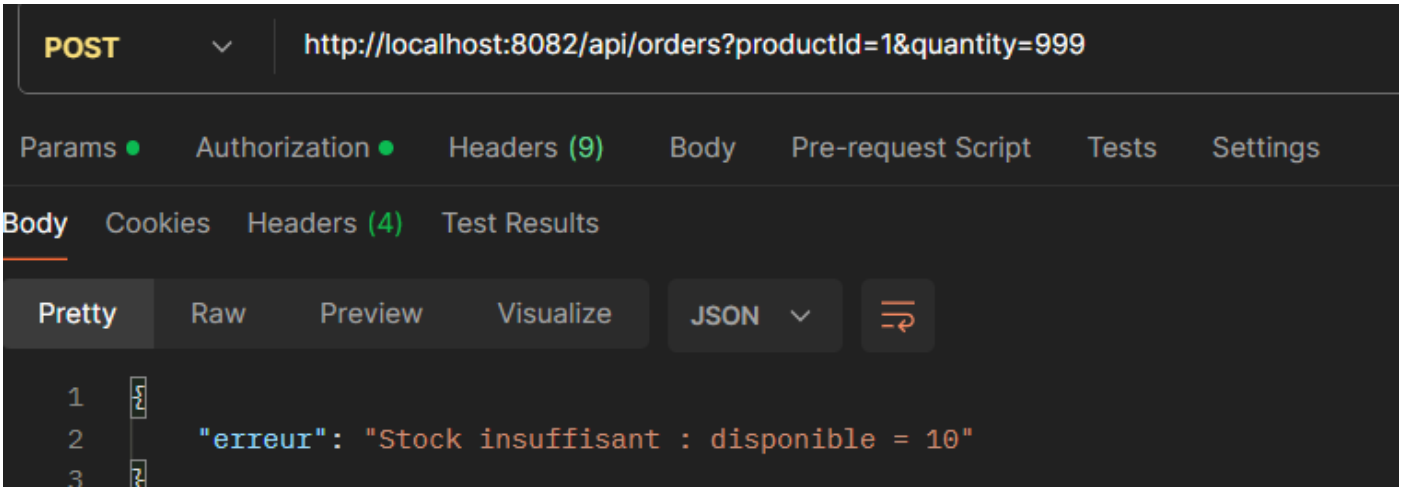
On crée GlobalExceptionHandler.java dans le dossier controller

```java
package com.ecommerce.orderservice.controller;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(RuntimeException.class)  no usages
    public ResponseEntity<Map<String, String>> handleRuntimeException(
            RuntimeException ex) {
        return ResponseEntity
                .status(HttpStatus.BAD_REQUEST)
                .body(Map.of( k1: "erreur", ex.getMessage()));
    }
}
```

Test produit inexistant :



test stock insuffisant :



Conclusion

L'implémentation de cette architecture SOA avec Spring Boot a permis de comprendre comment des microservices indépendants peuvent communiquer et collaborer efficacement via Eureka, Feign et une API Gateway, tout en garantissant la robustesse du système grâce à une gestion appropriée des erreurs.